ITA06 – MACHINE LEARNING

Assignment 3 – CO3: Bayesian and Computational Learning

NAME: ASHIRA BRIDGET SUSAN S

REGISTER NUMBER: 192524316

COURSE: ITA0604 – MACHINE LEARNING

*All numbers in this report are produced by the single script ml_assignment3.py (Appendix; global seed 42, all random draws via numpy default_rng). Libraries are used only for verification (scikit-learn Naïve Bayes), plotting and the LP shattering check; Bayes, Naïve Bayes, belief-network inference, EM, bounds and learners are written from scratch. Notation: H/N = High/Normal temperature, Hv/Lt = Heavy/Light load, H/L = High/Low vibration; Ø = empty hypothesis; *?* = any value.*

Q1. Bayes theorem and concept learning (5 marks)

1.1 Bearing screening with Bayes theorem

$P(F \mid +) = P(+|F)P(F) / [P(+|F)P(F) + P(+|\neg F)P(\neg F)] = 0.95 \cdot 0.03 / (0.95 \cdot 0.03 + 0.08 \cdot 0.97) = 0.0285 / 0.1061 =$ **0.2686**.

Two independent positives: $P(F \mid +,+) = 0.95^2 \cdot 0.03 / (0.95^2 \cdot 0.03 + 0.08^2 \cdot 0.97) = 0.027075 / 0.033283 =$ **0.8135**.

Role of the prior. Only 3% of bearings are faulty, so even a good test produces many false alarms (0.08×0.97 = 7.8% of all bearings against 2.9% true detections); a single positive therefore leaves the bearing more likely healthy (27% faulty). The posterior is prior × likelihood ratio: a second independent test multiplies the odds by 0.95/0.08 ≈ 11.9 again. Changing the base rate alone changes the answer a lot: prior 1% → 0.107, 3% → 0.269, 10% → 0.569, 30% → 0.836 (single positive).

1.2 Find-S and Candidate-Elimination trace (D1–D6)

Ex.	Label	Find-S hypothesis	Specific boundary S	General boundary G
init		<Ø,Ø,Ø>	{<Ø,Ø,Ø>}	{<?,?,?>}
D1	+	<H,Hv,H>	{<H,Hv,H>}	{<?,?,?>}
D2	+	<H,Hv,?>	{<H,Hv,?>}	{<?,?,?>}
D3	–	<H,Hv,?>	{<H,Hv,?>}	{<H,?,?>, <?,Hv,?>}
D4	–	<H,Hv,?>	{<H,Hv,?>}	{<?,Hv,?>}
D5	+	<?,Hv,?>	{<?,Hv,?>}	{<?,Hv,?>}
D6	–	<?,Hv,?>	{<?,Hv,?>}	{<?,Hv,?>}

Find-S ignores negatives and generalises minimally on positives. Candidate-Elimination keeps both boundaries: D3 (–) specialises G to {<H,?,?>, <?,Hv,?>}; D4 (–) kills <H,?,?> (it covers D4) so G becomes {<?,Hv,?>} after pruning <H,Hv,?>; D5 (+) generalises S. Both algorithms converge to **<?, Heavy, ?>** ("fails early iff the load is heavy"); S = G, so the concept is fully identified.

1.3 Version space, brute-force MAP and a non-uniform prior (D1–D3)

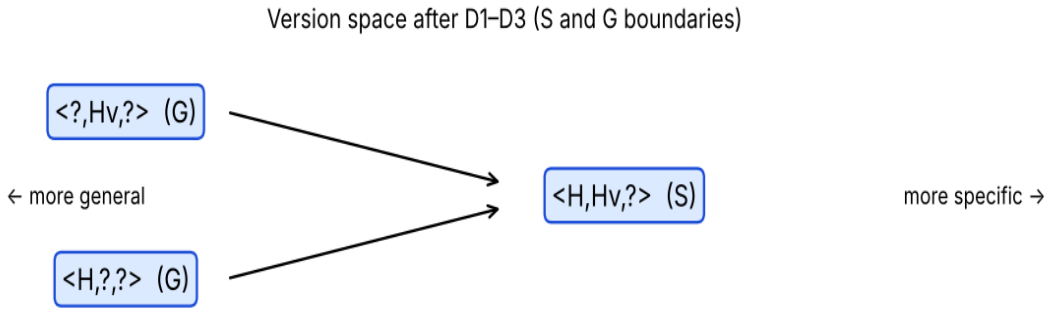


Fig. 1 – Version space after D1–D3: three hypotheses lie between S and G.

Enumerating all 3×3×3 + 1 hypotheses and keeping those consistent with D1–D3 gives exactly **three** hypotheses. With a noise-free likelihood P(D|h) = 1 if h is consistent and 0 otherwise, P(h|D) = P(h) / Σ P(h') over the version space.

Hypothesis	P(D|h)	Uniform prior	P(h|D) uniform	My prior	P(h|D) my prior
<H,Hv,?>	1	0.333	0.333	0.20	0.200
<H,?,?>	1	0.333	0.333	0.35	0.350
<?,Hv,?>	1	0.333	0.333	0.45	0.450

My prior (justified). P(<H,Hv,?>) = 0.20, P(<H,?,?>) = 0.35, P(<?,Hv,?>) = 0.45. The most specific hypothesis (covers only 2 of 8 instances) gets the smallest weight – an Occam-style preference for generality – and the load rule gets slightly more than the temperature rule because heavy load is the usual engineering driver of early bearing failure. **h_ML**: every hypothesis has likelihood 1, so ML is a three-way tie (it cannot choose). **h_MAP** = <?,Hv,?> (0.45) under my prior, whereas under the uniform prior MAP is also a three-way tie.

1.4 Reasoning inquiry

When do h_ML and h_MAP coincide? When the prior is uniform (P(h) constant, so argmax P(D|h)P(h) = argmax P(D|h)), or more generally when the prior does not reorder the hypotheses that have the highest likelihood. **A strong prior can overturn ML.** Use all six records with a noisy likelihood P(D|h) = (1–η)^(correct) η^(errors), η = 0.1. The numbers of errors are 1, 2, 0 for <H,Hv,?>, <H,?,?>, <?,Hv,?>, so P(D|h) = 0.0590, 0.0066, 0.5314 and **h_ML = <?,Hv,?>**. A mild

prior (0.20/0.35/0.45) keeps it (posterior 0.047/0.009/0.944). With a very strong prior 0.01 / 0.98 / 0.01 the posteriors become 0.048 / 0.521 / 0.431, so **h_MAP flips to <H,?,?>**, although it makes two training mistakes; the flip happens once the prior on <H,?,?> exceeds ≈ 0.976.

MAP versus the whole version space. For the new bearing x = (Normal, Heavy, Low) the three hypotheses predict No, No, Yes. Under my prior h_MAP = <?,Hv,?> says **Yes**, but the posterior-weighted vote is Yes = 0.45 vs No = 0.55, i.e. **No**. A single h_MAP uses only one hypothesis' opinion; the full posterior combines all of them, which is exactly the Bayes optimal idea of Q2.

Q2. MDL principle, Bayes optimal classifier and Gibbs algorithm (5 marks)

2.1 Minimum description length

Tree	Nodes	L(h) = 12·nodes	Errors	L(D|h) = errors·$\log_2 200$	L(h)+L(D|h)
A	6	72.00	8	61.15	**133.15**
B	25	300.00	1	7.64	307.64

$\log_2 200 = 7.644$ bits. **MDL prefers Tree A** (133.15 < 307.64 bits): its 7 extra errors cost only 53.51 bits, far less than the 228 bits saved by 19 fewer nodes.

Reversal point. Set $6c + 8\cdot\log_2 200 = 25c + 1\cdot\log_2 200 \Rightarrow c^* = 7\cdot\log_2 200 / 19 =$ **2.816 bits/node**. Below this node cost the larger Tree B wins; above it Tree A wins (12 bits is well above c*). **MAP link.** h_MAP = argmax P(D|h)P(h) = argmin[$-\log_2 P(D|h) - \log_2 P(h)$]. Thus $L(D|h) = -\log_2 P(D|h)$ (the cost of encoding the errors, optimal code length) and $L(h) = -\log_2 P(h)$ (a prior that assigns probability 2^(–L(h)), so every extra node makes a tree exponentially less probable). MDL is MAP under a length-based prior.

2.2 Bayes optimal classification and the Gibbs algorithm

Hypothesis	P(h|D)	Prediction on X	Weighted vote +	Weighted vote –
h1	0.30	+	0.30	
h2	0.25	–		0.25
h3	0.20	–		0.20
h4	0.15	+	0.15	
h5	0.10	–		0.10
Total	1.00		**0.45**	**0.55**

(i) **MAP:** h1 (0.30) is the most probable hypothesis, so MAP predicts +. (ii) **Bayes optimal:** argmax over classes of Σ P(h|D)·1[h predicts class] gives – (0.55) over + (0.45), so it predicts –. The two rules disagree.

Gibbs algorithm: draw one hypothesis h at random according to P(h|D) and use its prediction for X. If the true hypothesis is also drawn from the posterior, Gibbs errs when the two independent draws disagree: E[err_Gibbs] = 2·0.45·0.55 = **0.495**. The Bayes optimal classifier errs with probability min(0.45, 0.55) = **0.45**. Relationship: E[err_Gibbs] ≤ 2·E[err_BO] (0.495 ≤ 0.90 ✓) and here Gibbs > BO as expected. **Monte Carlo check** (200,000 draws, seed 42): Gibbs error = **0.4940**, Bayes optimal error = **0.4497**, matching the exact values.

2.3 Design inquiry

The Bayes optimal label is a weighted vote, so the function "X ↦ –" realised on all inputs may be a combination (e.g. a majority of h1…h5) that is not equal to any single member of H – it lives in a richer space of vote functions. It is rarely used for large H because it needs P(h|D) and the prediction of **every** hypothesis for each query (exponential or infinite in size). The Gibbs algorithm needs only one posterior sample (e.g. by MCMC) per prediction, so it is cheap, and it still guarantees an expected error of at most twice the optimal one.

Q3. Naïve Bayes classifier – loan approval (6 marks)

Assumption: features are conditionally independent given the class, $P(x_1,x_2,x_3|c) = \prod_j P(x_j|c)$.
Decision rule: $\hat{c}$ = argmax_c [log P(c) + Σ_j log $P(x_j|c)$]. Priors: P(Yes) = 5/10 = 0.5, P(No) = 0.5. Conditional probability tables (maximum likelihood, 5 records per class):

Feature	Value	P(·|Yes)	P(·|No)
Income	High / Medium / Low	2/5, 2/5, 1/5	1/5, 1/5, 3/5
Employment	Salaried / Self-employed	4/5, 1/5	1/5, 4/5
Credit history	Good / Poor	4/5, 1/5	1/5, 4/5

3.1 Manual classification

Application	Score Yes	Score No	P(Yes|x)	P(No|x)	Class
(Low, Self-employed, Poor)	0.5·⅕·⅕·⅕ = 0.004	0.5·⅗·⅘·⅘ = 0.192	0.020	0.980	**No**
(Medium, Salaried, Good)	0.5·⅖·⅘·⅘ = 0.128	0.5·⅕·⅕·⅕ = 0.004	0.970	0.030	**Yes**

3.2 Unseen value "Very High" and Laplace smoothing

For x = (Very High, Salaried, Good), the ML estimate gives P(Very High | Yes) = P(Very High | No) = 0/5 = 0, so **both scores are exactly 0** (0.0, 0.0) and the posterior is 0/0 – undefined. One unseen value vetoes all other evidence (zero-frequency problem). **Add-α smoothing:** P(v|c) = (count + α)/(N_c + α·K), with K = 4 income values (3 seen + Very High). I use **α = 1** (standard Laplace; the data are tiny, so a mild correction is enough): P(Very High|c) = 1/9 for both classes, and the decision is now carried by Employment and Credit: P(Yes|x) = **0.862**.

α	P(Very High | class)	Score Yes	Score No	P(Yes | x)
0.1	0.019	0.0058	0.0004	0.933
1	0.111	0.0283	0.0045	0.862
5	0.200	0.0360	0.0160	0.692

Effect: small α stays close to the data (α = 0.1: 0.933), large α over-smooths all tables towards uniform and drags the posterior to 0.5 (α = 5: 0.692).

3.3 Implementation and verification

The class CatNB (Appendix) stores log-priors and log-CPTs, adds log-probabilities and normalises with the log-sum-exp trick. On the three applications above it agrees with scikit-learn's CategoricalNB(alpha=1) to **2.2e-16** (max absolute difference of posteriors). **Why log space:** a product of 2,000 probabilities of 0.3 underflows to 0.0 in floating point, while its log is a harmless -2407.95; with many features the un-logged score of every class becomes 0.

3.4 Redundant-feature experiment

Data (fixed seed 42, 2,000 records, 70/30 split = 1400 train / 600 test): y ~ Bernoulli(0.5); four informative binary features with $P(x_j=1|y)$ = (0.25/0.75, 0.40/0.60, 0.40/0.60, 0.45/0.55); x5 and x6 are copies of x1 with 5% random flips (correlation(x1,x5) = 0.87). Naïve Bayes (α = 1) is fitted with and without x5, x6.

Model	Test accuracy	Test log-loss	Mean predicted confidence
NB, 4 features (no redundancy)	0.752	0.518	0.736
NB, 6 features (x5, x6 redundant)	0.748	0.686	0.924

Interpretation. Violating independence did **not** reduce accuracy (0.752 → 0.748, within sampling noise) but increased log-loss by 32% and pushed the mean confidence from 0.74 to 0.92 although accuracy stayed at 0.75: x1's evidence is counted three times, so the log-odds are inflated and the probabilities are over-confident. The argmax is often unchanged because the inflation is monotone in the same direction as the true evidence. Hence an independence violation does not **always** hurt

accuracy, but it damages the reliability of predicted probabilities (see Fig. 5b), which matters whenever the probabilities feed a decision or a cost.

Q4. Bayesian belief network – production-line alarm (6 marks)

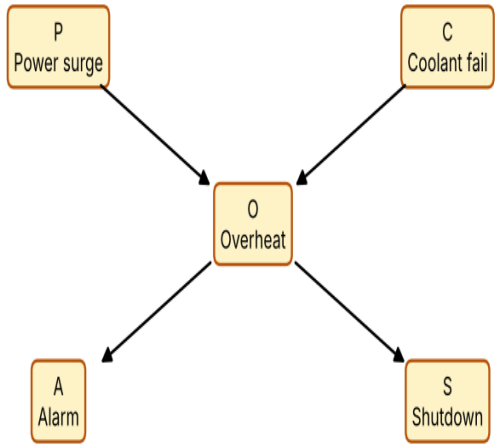


Fig. 2 – Belief network

Factored joint: P(P,C,O,A,S) = P(P)·P(C)·P(O|P,C)·P(A|O)·P(S|O).

Parameters: 1 + 1 + 4 + 2 + 2 = **10** independent numbers, versus $2^5 - 1$ = **31** for the full joint over five binary variables – a 3× saving that grows exponentially with the number of variables.

d-separation: a path is blocked by an observed non-collider/chain node, or by an unobserved collider whose descendants are all unobserved. O is a collider on P→O←C; A, S are children of O.

Statement	Holds ?	Justification
A ⫫ S | O	Yes	Only path A←O→S is a fork through O; observing O blocks it.
P ⫫ C (no evidence)	Yes	Only path P→O←C has the collider O unobserved (and A, S unobserved) → blocked.
P ⫫ C | O	No	Observing the collider O opens the path (explaining away).
P ⫫ C | A	No	A is a descendant of the collider O; observing it also activates the path.

4.1 Manual inference

P(O=T) = Σ_{p,c} P(p)P(c)P(O|p,c) = 0.1·0.05·0.95 + 0.1·0.95·0.60 + 0.9·0.05·0.80 + 0.9·0.95·0.02 = 0.00475 + 0.05700 + 0.03600 + 0.01710 = **0.1149**.

P(C=T | A=T): P(O|C=T) = 0.1·0.95 + 0.9·0.80 = 0.815; P(O|C=F) = 0.1·0.60 + 0.9·0.02 = 0.078. P(A|C=T) = 0.815·0.9 + 0.185·0.05 = 0.74275; P(A|C=F) = 0.078·0.9 + 0.922·0.05 = 0.1163. P(C|A) = 0.05·0.74275 / (0.05·0.74275 + 0.95·0.1163) = 0.03714 / 0.14762 = **0.2516**.

P(C=T | A=T, P=T): P(A|P,C=T) = 0.95·0.9 + 0.05·0.05 = 0.8575; P(A|P,C=F) = 0.60·0.9 + 0.40·0.05 = 0.56. P(C|A,P) = 0.05·0.8575 / (0.05·0.8575 + 0.95·0.56) = 0.042875 / 0.574875 = **0.0746**.

Explaining away. P(C=T|A=T) = 0.2516 is five times the prior 0.05 – an alarm points to overheating, whose cause is either a surge or a coolant failure. Once we also know P = T, the surge already explains

the overheating (and so the alarm), so the evidence for coolant failure drops to 0.0746, almost back to its prior. Two independent causes become competing explanations of a shared effect.

4.2 Inference by enumeration (coded) – verifies all manual answers

Query	Coded value	Joint terms summed
P(O=T)	0.1149	32
P(C=T|A=T)	0.2516	16
P(C=T|A=T,P=T)	0.0746	8
P(C=T)	0.0500	32
P(C=T|O=T)	0.3548	16
P(C=T|P=T)	0.0500	16

Terms = joint entries (out of 32) summed over both values of the query variable: the evidence fixes some variables and the remaining hidden variables are enumerated. P(C=T|O=T) = 0.355 and P(C=T|P=T) = 0.050 confirm the d-separation answers (O couples P and C; P alone does not change C). The manual calculations used 4 – 8 terms because A and S were summed out analytically.

4.3 Learning CPTs from simulated data (1,000 records, seed 42)

Sample size	TT (true 0.95)	TF (true 0.60)	FT (true 0.80)	FF (true 0.02)	Mean abs. error
50	n/a (0 obs)	1.000 (n=3)	0.500 (n=2)	0.022 (n=45)	0.234 *
200	1.000 (n=1)	0.882 (n=17)	0.800 (n=5)	0.011 (n=177)	0.085
1000	1.000 (n=7)	0.602 (n=88)	0.903 (n=31)	0.016 (n=874)	0.040

(* error over the defined cells only.) The ML estimate of P(O=T|P,C) = (count of O=T)/(count of the parent combination). The error falls from 0.234 to 0.040 as the sample grows, but the rare parent configuration (P=T, C=T has probability 0.005) has no observation at n = 50 and only 7 at n = 1,000 – small samples give unreliable or undefined CPT rows (this motivates smoothing/priors).

4.4 Design inquiry

Naïve Bayes assumes **all features are conditionally independent given the class** and has a fixed star structure. The belief network relaxes this: variables may depend on other variables (P and C are marginally independent but become dependent given O), so the structure encodes exactly the independences that hold. The price: exact inference by enumeration sums $O(2^n)$ terms; variable elimination helps for tree-like graphs, but in general inference is NP-hard and large or densely connected networks need approximation (sampling, variational).

Q5. EM algorithm for a two-Gaussian mixture (7 marks)

Model: $p(x) = \Sigma_{k=1}^{2} \pi_k N(x \mid \mu_k, \sigma_k^2)$, $\Sigma\pi_k = 1$. **Hidden variable:** $z_i \in \{1,2\}$, the (unrecorded) machine that produced reading x_i. **E-step:** $r_{ik} = P(z_i=k|x_i) = \pi_k N(x_i|\mu_k,\sigma_k^2) / \Sigma_j \pi_j N(x_i|\mu_j,\sigma_j^2)$. **M-step:** $N_k = \Sigma_i r_{ik}$; $\pi_k = N_k/N$; $\mu_k = \Sigma_i r_{ik} x_i / N_k$; $\sigma_k^2 = \Sigma_i r_{ik} (x_i - \mu_k)^2 / N_k$.

Derivation of μ_k. The expected complete-data log-likelihood is $Q(\theta|\theta_{old}) = \Sigma_i \Sigma_k r_{ik} [\ln \pi_k - \tfrac{1}{2} \ln(2\pi\sigma_k^2) - (x_i - \mu_k)^2/(2\sigma_k^2)]$. Setting $\partial Q/\partial \mu_k = \Sigma_i r_{ik} (x_i - \mu_k)/\sigma_k^2 = 0$ gives $\Sigma_i r_{ik} x_i = \mu_k \Sigma_i r_{ik}$, i.e. **$\mu_k = \Sigma_i r_{ik} x_i / \Sigma_i r_{ik}$** – a responsibility-weighted mean. (Likewise $\partial Q/\partial \sigma_k^2 = 0$ gives the variance update, and a Lagrange multiplier for $\Sigma\pi_k = 1$ gives $\pi_k = N_k/N$.)

5.1 One EM iteration by hand (7 readings; $\pi = 0.5$, $\mu = (2, 8)$, $\sigma^2 = (1, 1)$)

x	1.8	2.5	3.1	5.0	7.6	8.4	9.0
r_1 (machine 1)	1.0000	1.0000	1.0000	0.5000	0.0000	0.0000	0.0000
r_2 (machine 2)	0.0000	0.0000	0.0000	0.5000	1.0000	1.0000	1.0000

($x = 5.0$ is equidistant from both means, so $r = 0.5/0.5$.) Effective counts $N_1 = 3.500$, $N_2 = 3.500$ ⇒ **π = (0.500, 0.500)**; **μ = (2.829, 7.857)** (e.g. $\mu_1 = \Sigma\, r_{i1}\, x_i / N_1 = 9.90/3.50$); **$\sigma^2$ = (1.028, 1.643)**. **Log-likelihood before = -16.50, after = -14.98** – it increased, as EM guarantees.

5.2 Implementation, monotone log-likelihood and convergence rule

Data: 500 readings (seed 42) from $\pi = (0.4, 0.6)$, $\mu = (2, 6)$, $\sigma^2 = (1, 2.25)$. **Convergence rule:** stop when $|LL_t - LL_{t-1}| < 10^{-6}$ or after 1,000 iterations; variances are floored at 10^{-6} to avoid singularities. The log-likelihood was recorded at every iteration of all 10 runs below and **never decreased** (check monotone = True for every run; Fig. 3a).

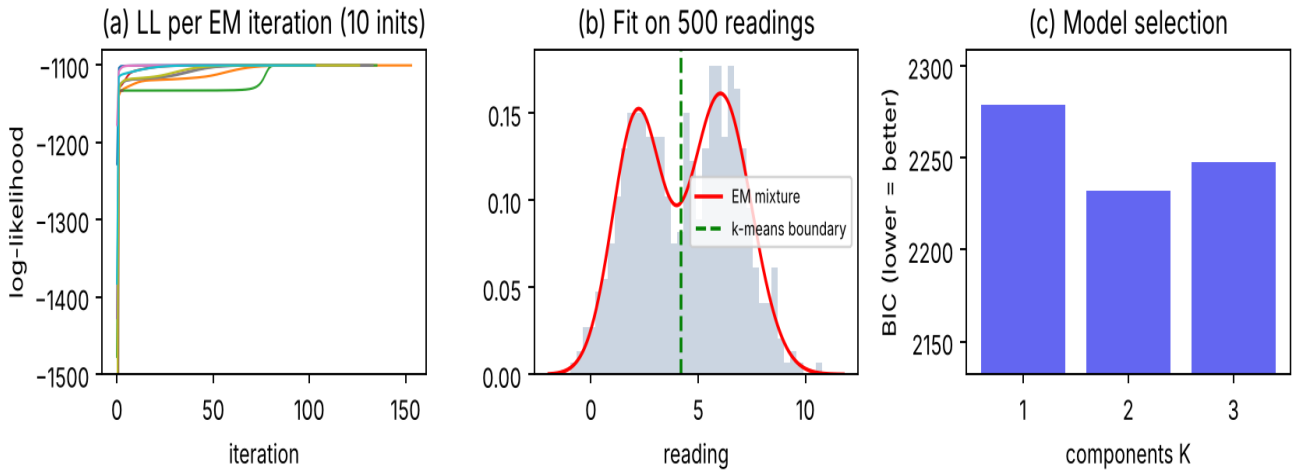


Fig. 3 – (a) log-likelihood per iteration for 10 random initialisations, (b) fitted mixture and k-means boundary, (c) BIC for K = 1, 2, 3.

5.3 Ten random initialisations

Run	Init μ	Final LL	π_1, π_2	μ_1, μ_2	σ_1^2, σ_2^2	Iters	Result
0	6.5, 2.2	-1100.50	0.57, 0.43	6.06, 2.15	2.01, 1.32	77	global, labels swapped
1	5.0, 10.2	-1100.50	0.43, 0.57	2.15, 6.06	1.32, 2.01	153	global
2	2.1, 2.5	-1100.50	0.43, 0.57	2.15, 6.06	1.32, 2.01	135	global
3	0.0, 1.8	-1100.50	0.43, 0.57	2.15, 6.06	1.32, 2.01	101	global
4	10.1, 5.0	-1100.50	0.57, 0.43	6.06, 2.15	2.01, 1.32	131	global, labels swapped
5	8.5, 8.5	-1133.14	0.52, 0.48	4.35, 4.43	5.44, 5.44	3	**local optimum / plateau**
6	5.4, 3.1	-1100.50	0.57, 0.43	6.06, 2.15	2.01, 1.32	78	global, labels swapped
7	6.4, 9.6	-1100.50	0.43, 0.57	2.15, 6.06	1.32, 2.01	133	global
8	2.9, 10.6	-1100.50	0.43, 0.57	2.15, 6.06	1.32, 2.01	126	global
9	9.2, 2.4	-1100.50	0.57, 0.43	6.06, 2.15	2.01, 1.32	103	global, labels swapped

Nine runs reach the same maximum (LL = –1100.50; $\pi \approx (0.43, 0.57)$, $\mu \approx (2.15, 6.06)$, $\sigma^2 \approx (1.32, 2.01)$ – close to the truth (0.4/0.6, 2/6, 1/2.25)). Run 5 stops at LL = –1133.14 with both means ≈ 4.4: a plateau/saddle that equals the one-component fit; the tolerance rule fired after 3 iterations, so the **stopping rule can falsely signal convergence on a flat region**. Runs 0, 4, 6, 9 returned the same solution with components swapped ($\mu_1 > \mu_2$): **label switching** – a relabelling, not a different optimum.

5.4 Choosing the number of components

K	Parameters	Log-likelihood	BIC = p·ln N – 2LL
1	2	-1133.14	2278.71
2	5	-1100.50	**2232.07**
3	8	-1098.99	2247.70

Going from K = 1 to 2 raises LL by 32.6 (large), while K = 3 gains only 1.5 LL for 3 extra parameters, far less than the BIC penalty (3·ln 500 = 18.6). **BIC selects K = 2**, matching the generating process.

5.5 Soft (EM) versus hard (k-means style) assignment

Method	Means	Mixing weights	Variances	LL (Gaussian model)	Accuracy vs true labels
EM (soft)	2.15, 6.06	0.43, 0.57	1.32, 2.01	-1100.50	0.928
k-means (hard)	2.18, 6.23	0.46, 0.54	1.19, 1.54	-1104.24	0.920

Hard assignment cuts the overlap region at one boundary (Fig. 3b), so each cluster's mean/variance are biased (means pushed away from the overlap, weights 0.46/0.54 vs. true 0.40/0.60) and its likelihood is lower by 3.7. EM keeps uncertainty: 110 of 500 points have a largest responsibility below 0.9, information that k-means discards.

5.6 Analysis inquiry

Why LL never decreases. For any distribution q over z, $LL(\theta) = F(q,\theta) + KL(q\|p(z|x,\theta)) \geq F(q,\theta)$, where $F = E_q[\ln p(x,z|\theta)] + H(q)$. The E-step sets $q = p(z|x,\theta_t)$, making KL = 0 so $F(q,\theta_t) = LL(\theta_t)$; the M-step maximises F over θ, so $LL(\theta_{t+1}) \geq F(q,\theta_{t+1}) \geq F(q,\theta_t) = LL(\theta_t)$. **Why no global guarantee:** this only climbs a bound to a stationary point; the mixture likelihood is non-convex, so the limit depends on the start (run 5). **Relation to ML:** EM is an algorithm for **maximum-likelihood estimation when the likelihood involves hidden variables** – it maximises the marginal likelihood $p(x|\theta) = \Sigma_z p(x,z|\theta)$. **CPTs of the Q4 network with unobserved variables:** if, say, O is not recorded, the E-step computes the posterior P(O | P, C, A, S) for each record by enumeration (Q4) and the M-step re-estimates each CPT from **expected counts**, e.g. $P(O=T|P=p,C=c) = \Sigma_i P(O_i=T | e_i)\cdot 1[P_i=p, C_i=c] / \Sigma_i 1[P_i=p, C_i=c]$; repeat until the log-likelihood converges.

Q6. Sample complexity, hypothesis-space capacity and mistake bounds (8 marks)

Part A – PAC learning with a finite hypothesis space

PAC framework (ε, δ): a learner is PAC if, for every target concept in C, every distribution D and every $\varepsilon, \delta \in (0,1)$, it outputs with probability $\geq 1 - \delta$ a hypothesis with true error $\leq \varepsilon$, using a number of examples (and time) polynomial in $1/\varepsilon$, $1/\delta$, n. For a consistent learner in finite H: $m \geq (1/\varepsilon)(\ln|H| + \ln(1/\delta))$. For conjunctions, $|H| = 3^n + 1 = 59{,}050$ for n = 10 ($\ln|H| = 10.99$).

	δ = 0.05	δ = 0.01
ε = 0.10	140	156
ε = 0.05	280	312

Required sample sizes (n = 10). **Strongest influence:** ε – m ∝ 1/ε, so reducing ε tenfold multiplies m by ten; n enters linearly (each extra feature adds ln3/ε ≈ 22 examples at ε = 0.05); δ enters only logarithmically (making δ 100× smaller adds ln(100)/ε ≈ 92 examples).

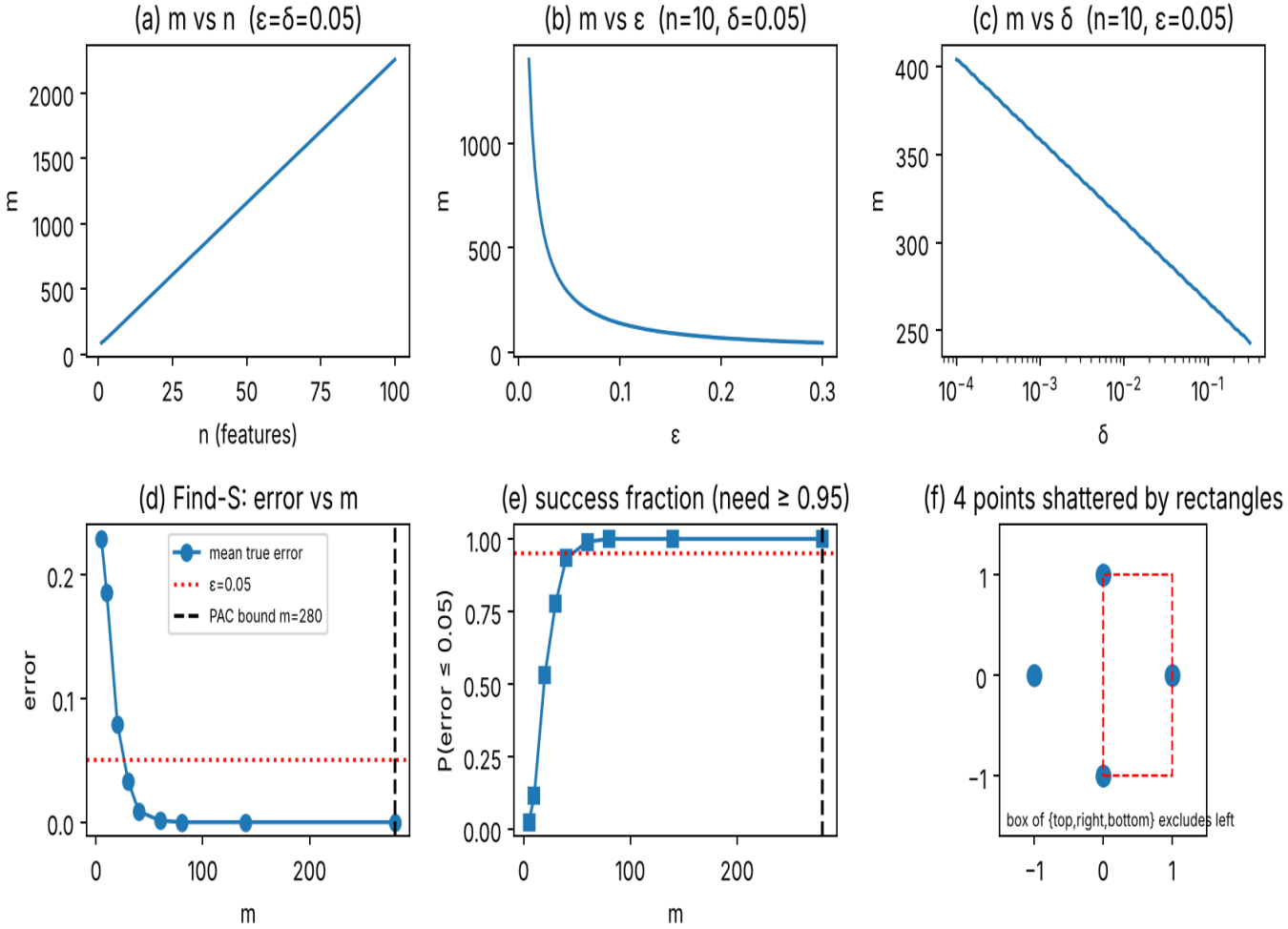


Fig. 4 – (a–c) PAC bound against n, ε and δ; (d, e) Find-S simulation; (f) four points shattered by axis-aligned rectangles.

Find-S simulation. Target: $x_1 \wedge \neg x_2$ over n = 10 Boolean features; examples uniform on $\{0,1\}^{10}$, noise-free; 500 trials for each sample size (seed 42); true error computed exactly (error = P(target) – P(h)). At the PAC bound for ε = δ = 0.05 (m = 280) the mean true error is 0.0000 and **100% of trials** have error ≤ 0.05 (required ≥ 95%). Already at m = 40 the success fraction is 0.934 and at m = 60 it is 0.990. The bound is therefore **very conservative** (~5–7× too many samples here): it covers the worst target and the worst distribution, and uses the crude union bound over all 59,050 hypotheses.

Part B – Infinite hypothesis spaces: shattering and VC dimension

A set S of points is **shattered** by H if for every one of the 2^|S| labellings some h ∈ H realises it; **VC(H)** is the size of the largest shattered set. Verified by code where marked ✓.

Class	VC	Shattered configuration	Why nothing larger is shattered
Intervals on $\mathbb{R}$	2	Points {1, 2}: [1,2] → ++, [1,1] → +–, [2,2] → –+, empty → — ✓	For $x_1<x_2<x_3$ the labelling + – + needs a non-convex set.
Linear separators in $\mathbb{R}^2$	3	Three non-collinear points (e.g. (0,0), (1,0), (0,1)): all 8 labellings are linearly separable ✓ (LP check)	The XOR labelling of a square's corners is not separable ✓ (also for any 4 points, a point inside the hull or crossing diagonals).
Axis-aligned rectangles	4	Diamond (0,1), (1,0), (0,–1), (–1,0): the bounding box of any subset excludes the others ✓ (Fig. 4f)	For 5 points, the extreme left/right/top/bottom points (+) force a box containing the fifth (–) ✓.

VC sample bound $m \geq (1/\varepsilon)(4 \log_2(2/\delta) + 8 \cdot VC \cdot \log_2(13/\varepsilon))$ with $\varepsilon = 0.1$, $\delta = 0.05$: intervals **1337**, linear separators **1899**, rectangles **2461** (conjunctions with VC = 10 would need ≈ 5831 vs 140 from the finite-space bound). **Nature of the bound:** it depends on the capacity (VC) rather than on |H|, so it applies to infinite classes where $\ln|H| = \infty$; it grows linearly with VC and has larger constants (it must hold for every distribution and every class of that VC dimension), so it is more conservative than ln|H|-based bounds when |H| is finite.

Part C – Mistake-bound model

Halving algorithm. H = all 64 monotone conjunctions over 6 Boolean variables ($|H| = 2^6$). The learner predicts the majority vote of the hypotheses consistent so far and deletes every inconsistent one, so each mistake at least halves the version space: mistakes ≤ $\log_2|H| = 6$. Over 2,000 random targets × 100 random examples (seed 42): **mean 2.00 mistakes, maximum 5 ≤ 6** ✓.

Online conjunction learner (n = 10). Start with the most specific hypothesis containing all 2n literals (it predicts negative for everything); on a mistake (always a positive example) delete every literal the example falsifies. The first mistake removes n literals (the n false ones among the 2n); each later mistake removes at least one of the remaining ≤ n literals, so mistakes ≤ n + 1 = 11. Over 3,000 random sequences (random targets, several example distributions): **max 11, mean 5.39** ✓; an adversarial sequence (all-ones, then one zero at a time) achieves exactly **11 = n + 1**, so the bound is tight.

$VC(C) \leq Opt(C) \leq \log_2|C|$. Class C = thresholds {x ≥ t}, t = 1…5, on X = {1,2,3,4}: |C| = 5, $\log_2 5 = 2.32$. Brute force gives VC = **1** (one point can be shattered, two cannot: (+,–) in order is impossible), and a minimax recursion over the version space gives Opt = **2**, so **1 ≤ 2 ≤ 2.32** ✓. VC ≤ Opt because

an adversary can force a mistake on each of d shattered points; Opt ≤ $\log_2|C|$ because Halving attains it.

Selection inquiry

PAC counts the **training examples** needed so that a hypothesis learned from i.i.d. data is probably approximately correct on future draws; the mistake-bound model counts **errors made while learning online**, for any (even adversarial) sequence and with no ε, δ and no distribution assumption. For the quality team: if historical, i.i.d. inspection records are labelled in a campaign, the PAC bound helps budget the labelling (as an upper limit – here it was 5–7× pessimistic). If the learner is deployed on the line and each missed defect has a cost, the mistake bound is more relevant (at most n + 1 mistakes). **Assumptions in my simulations:** noise-free labels, a realisable target (a conjunction in H), uniformly distributed independent binary features, i.i.d. examples, a known number n of features; with label noise or a non-conjunctive target the guarantees do not apply.

Q7. Day-4 design challenge – diagnose, redesign and validate (13 marks)

A. Diagnosis (selected solution: Naïve Bayes of Q3)

Limitation: with redundant features Naïve Bayes keeps its accuracy but produces **unreliable (over-confident) probabilities**. **Evidence (Q3):** adding two near-copies of x1 (correlation 0.87) changes accuracy 0.752 → 0.748 but raises test log-loss 0.518 → 0.686 (+32%) and mean confidence 0.74 → 0.92; the reliability curve (Fig. 5b, red) falls far below the diagonal at high confidence (predicted 0.95, observed ≈ 0.76). **Source:** the model assumption – not limited data, the prior, initialisation or inference cost – since removing the two redundant columns on the same data restores log-loss (0.518). Mechanism: x1's log-likelihood ratio is counted three times.

B. Design objective

Baseline: NB on 6 features, test log-loss 0.686 (seed 42); 0.735 ± 0.056 over 20 further seeds. **Target:** reduce log-loss by at least 10% (≤ 0.617 on seed 42, and in the 20-seed mean) **while** accuracy drops by at most 1 percentage point and the parameter count stays below 2× the baseline (13 → ≤ 26). Accepted trade-offs: higher training cost (milliseconds here) and more parameters.

C. Revised strategy – Tree-Augmented Naïve Bayes (TAN)

What changes and why. Keep the class-conditional factorisation but relax independence **only where the data show dependence**, using the belief-network machinery of Q4: every feature has the class as parent plus **at most one feature parent**. The parent is chosen by a maximum-weight spanning tree on the conditional mutual information $I(X_i;X_j|Y)$ (Chow–Liu), so the near-duplicate x5 is modelled as P(x5 | Y, x1) and no longer adds a second independent vote. CPTs use Laplace smoothing α = 1 (Q3). Inference stays exact and linear in the number of features: $P(y|x) \propto P(y) \prod_j P(x_j \mid y, x_pa(j))$. **Why not a general belief network?** It would need structure search and exponential inference; TAN needs only

$O(d^2N)$ for the CMI table and a spanning tree. **Cost:** parameters 13 → 23 (1 + 2 + 5·2·2), $O(d^2)$ CMI computations, and more data needed per CPT; only one parent per feature, so higher-order redundancy is not captured.

TAN(X, y, alpha):

1. for each pair (i, j): W[i,j] = sum_c P(c) * sum_{a,b} P(a,b|c) log[P(a,b|c) / (P(a|c)P(b|c))] # conditional MI
2. T = maximum spanning tree of W (Prim, root = x1); orient edges away from the root -> parent(j)
3. for each feature j: CPT[c, parent value, v] = (count + alpha) / (count of parent&c + alpha*K)
4. predict: log P(c) + sum_j log P(x_j | c, x_parent(j)) -> log-sum-exp -> posterior

D. Prediction (recorded before running TAN)

TAN should (i) link x5 and x6 to x1; (ii) bring log-loss to within ~5% of the NB that never saw the redundant columns (≈ 0.52), i.e. well above the 10% target; (iii) leave accuracy within ±1 point; (iv) bring the reliability curve back to the diagonal. Expected improving measure: log-loss and calibration, **not** accuracy.

E. Prototype comparison (same data, split and seeds as Q3)

Model	Params	Acc. (seed 42)	Log-loss (seed 42)	Acc. (20 seeds)	Log-loss (20 seeds)	Fit time
NB, 4 features	9	0.752	0.518	0.752 ± 0.016	0.529 ± 0.018	–
NB, 6 features (baseline)	13	0.748	0.686	0.753 ± 0.017	0.735 ± 0.056	0.17 ms
TAN, 6 features	23	0.752	**0.517**	0.750 ± 0.016	**0.532 ± 0.018**	1.72 ms

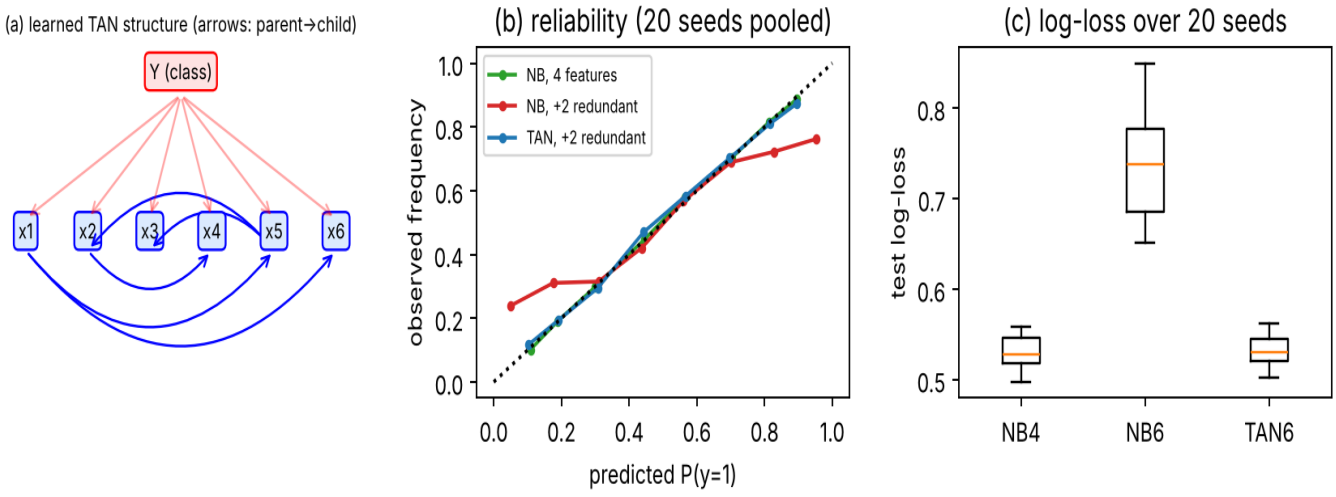


Fig. 5 – (a) learned TAN structure, (b) reliability diagram, (c) log-loss over 20 seeds.

F. Evaluation and final decision

Objective met. Log-loss fell by **24.6%** on seed 42 (0.686 → 0.517) and **27.5%** over 20 seeds (target ≥ 10%); accuracy changed by +0.3 pp (≤ 1 pp allowed); parameters 13 → 23 (< 26). **Prediction check:** the structure has x5←x1 and x6←x1 ✓; TAN log-loss is within 0.6% of NB4 ✓; reliability returns to the diagonal ✓; accuracy unchanged ✓. The learned tree also contains weak, spurious edges (x2←x5, x3←x5, x4←x2) – harmless here but a sign of the extra variance that a one-parent model with 1,400 records can pick up. **New costs:** about 10× the fit time (still ~1.7 ms), ten more parameters, and a structure-learning step that needs enough data to estimate pairwise dependence.

When the original remains preferable: TAN does not beat NB trained without the duplicates (0.532 vs 0.529), so if the redundant columns are known, simply dropping them is cheaper and just as good. Plain NB is also preferable when only the ranking/label is needed (accuracy is equal), with very small samples (extra parameters), or when features are close to independent. **Final decision:** use TAN whenever calibrated probabilities are needed and redundancy cannot be ruled out in advance; otherwise keep NB with feature screening.

Acknowledgement. *Course notes (ITA06) and standard references on Naïve Bayes, EM and PAC theory; AI assistance (Claude) was used to draft the code and report, and all results were regenerated by running the submitted script.*

Appendix – Source code (ml_assignment3.py)

Run with python ml_assignment3.py; it writes results.json and the figures in figs/. Seeds: global 42; Q3/Q7 data seeds 42 (main) and 100–119 (robustness); EM restarts use seeds 0–9 (and 100–109 for model selection).

```
"""ITA06 Machine Learning - Assignment 3 (CO3). Run: python ml_assignment3.py
Every number in the report is produced here (global seed 42 unless stated)."""
import itertools, json, math, os
import numpy as np
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.optimize import linprog

HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, "figs"); os.makedirs(OUT, exist_ok=True)
SEED = 42
R = {}                              # results store -> results.json
def save(name):
    plt.tight_layout(); plt.savefig(f"{OUT}/{name}.png", dpi=170); plt.close()

# ================================ Q1
==========================================
def posterior(prior, tpr, fpr, k):       # P(faulty | k independent positives)
    num = prior * tpr**k
    return num / (num + (1 - prior) * fpr**k)
R["q1_bayes"] = [posterior(.03, .95, .08, k) for k in (1, 2)]
R["q1_prior_sweep"] = {p: posterior(p, .95, .08, 1) for p in (.01, .03, .10, .30)}

ATTR = [("High", "Normal"), ("Heavy", "Light"), ("High", "Low")]
D = [(("High", "Heavy", "High"), 1), (("High", "Heavy", "Low"), 1), (("Normal", "Light", "Low"), 0),
     (("High", "Light", "High"), 0), (("Normal", "Heavy", "High"), 1), (("Normal", "Light", "High"), 0)]
AB = {"High": "H", "Normal": "N", "Heavy": "Hv", "Light": "Lt", "Low": "L", "?": "?", "0": "∅"}
fmt = lambda h: "<" + ",".join(AB[a] for a in h) + ">"
covers = lambda h, x: all(a == "?" or a == b for a, b in zip(h, x))     # "0" = empty constraint
more_general = lambda h1, h2: all(a == "?" or a == b or b == "0" for a, b in zip(h1, h2))

def find_s(D):
```

```
    s, tr = ["0"] * 3, []
    for x, y in D:
        if y:
            for i, v in enumerate(x):
                s[i] = v if s[i] == "0" else (s[i] if s[i] == v else "?")
        tr.append(fmt(s))
    return tr

def cand_elim(D):
    S, G, tr = [("0",) * 3], [("?",) * 3], []
    for x, y in D:
        if y:
            G = [g for g in G if covers(g, x)]
            nS = []
            for s in S:
                if covers(s, x): nS.append(s); continue
                m = tuple(v if si == "0" else (si if si == v else "?") for si, v in zip(s, x))
                if any(more_general(g, m) for g in G): nS.append(m)
            S = nS
        else:
            S = [s for s in S if not covers(s, x)]
            nG = []
            for g in G:
                if not covers(g, x): nG.append(g); continue
                for i in range(3):
                    if g[i] == "?":
                        for v in ATTR[i]:
                            if v != x[i]:
                                h = g[:i] + (v,) + g[i + 1:]
                                if any(more_general(h, s) for s in S): nG.append(h)
            nG = list(dict.fromkeys(nG))
            G = [g for g in nG if not any(g != o and more_general(o, g) for o in nG)]
        tr.append(([fmt(s) for s in S], [fmt(g) for g in G]))
    return tr
R["q1_finds"], R["q1_ce"] = find_s(D), cand_elim(D)
```

```
HYP = list(itertools.product(("High", "Normal", "?"), ("Heavy", "Light", "?"), ("High", "Low", "?")))
+ [("0",) * 3]
VS = [h for h in HYP if all(covers(h, x) == bool(y) for x, y in D[:3])]
R["q1_vs"] = [fmt(h) for h in VS]
def post(prior, lik):
    z = sum(p * l for p, l in zip(prior, lik)); return [p * l / z for p, l in zip(prior, lik)]
uni, mine = [1 / 3] * 3, [.20, .35, .45]            # order of VS: <H,Hv,?>, <H,?,?>, <?,Hv,?>
R["q1_post_uniform"] = post(uni, [1] * 3)
R["q1_post_mine"] = post(mine, [1] * 3)
eta = .1
def lik_noisy(h, data):
    err = sum(covers(h, x) != bool(y) for x, y in data); return (1 - eta) ** (len(data) - err) * eta ** err
L6 = [lik_noisy(h, D) for h in VS]
R["q1_L6"] = L6
R["q1_noisy_uniform"] = post(uni, L6)
R["q1_noisy_mine"] = post(mine, L6)
R["q1_noisy_strong"] = post([.01, .98, .01], L6)
for p in np.arange(.34, 1, .001):                   # smallest prior on <H,?,?> that flips MAP
    if np.argmax(post([(1 - p) / 2, p, (1 - p) / 2], L6)) == 1: R["q1_flip_p"] = float(p); break
xnew = ("Normal", "Heavy", "Low")
pred = [int(covers(h, xnew)) for h in VS]
R["q1_newx"] = {"pred": pred, "vote_uniform": sum(p * q for p, q in zip(R["q1_post_uniform"],
pred)),
                "vote_mine": sum(p * q for p, q in zip(R["q1_post_mine"], pred))}


fig, ax = plt.subplots(figsize=(6.6, 1.9)); ax.axis("off")
pos = {"G1": (.12, .5), "G2": (.5, .5), "S": (.88, .5)}
nodes = [("<?,Hv,?>  (G)", .12, .78), ("<H,?,?>  (G)", .12, .22), ("<H,Hv,?>  (S)", .62, .5)]
for t, x, y in nodes: ax.text(x, y, t, ha="center", va="center", bbox=dict(boxstyle="round",
fc="#dbeafe", ec="#1d4ed8"))
for y in (.78, .22): ax.annotate("", (.5, .54 if y > .5 else .46), (.22, y), arrowprops=dict(arrowstyle="-
>"))
ax.text(.9, .5, "more specific →", va="center", fontsize=8); ax.text(.0, .5, "← more general",
va="center", fontsize=8, ha="left")
ax.set_xlim(0, 1); ax.set_ylim(0, 1); ax.set_title("Version space after D1–D3 (S and G boundaries)",
fontsize=9); save("q1_vs")
```

```
# ================================ Q2
================================================
L = math.log2(200); dl = lambda nodes, err, c=12: nodes * c + err * L
R["q2"] = {"log2_200": L, "A": [6 * 12, 8 * L, dl(6, 8)], "B": [25 * 12, 1 * L, dl(25, 1)], "c_star": 7 *
L / 19}
post_h = np.array([.30, .25, .20, .15, .10]); pr = np.array([1, 0, 0, 1, 0])      # 1 = '+'
R["q2"]["vote"] = {"+": float(post_h[pr == 1].sum()), "-": float(post_h[pr == 0].sum())}
rng = np.random.default_rng(SEED); N = 200_000
true = rng.choice(5, N, p=post_h); gibbs = rng.choice(5, N, p=post_h)
bo = int(R["q2"]["vote"]["+"] > R["q2"]["vote"]["-"])
R["q2"]["gibbs_exact"] = 2 * .45 * .55; R["q2"]["bo_exact"] = min(.45, .55)
R["q2"]["gibbs_mc"] = float((pr[gibbs] != pr[true]).mean()); R["q2"]["bo_mc"] = float((bo !=
pr[true]).mean())

# ================================ Q3
================================================
class CatNB:
    """Categorical Naive Bayes, integer-coded features, log-space, add-alpha smoothing."""
    def __init__(s, alpha=1.0, K=None): s.a, s.K = alpha, K
    def fit(s, X, y):
        s.cls = np.unique(y); d = X.shape[1]; s.K = s.K or [int(X[:, j].max()) + 1 for j in range(d)]
        s.prior = np.array([(y == c).mean() for c in s.cls])
        s.cpt = [np.stack([(np.bincount(X[y == c, j], minlength=s.K[j]) + s.a) / ((y == c).sum() + s.a *
s.K[j])
                           for c in s.cls]) for j in range(d)]
        return s
    def log_joint(s, X):
        with np.errstate(divide="ignore"):
            out = np.tile(np.log(s.prior), (len(X), 1))
            for j in range(X.shape[1]): out += np.log(s.cpt[j][:, X[:, j]]).T
        return out
    def predict_proba(s, X):
        lj = s.log_joint(X); m = lj.max(1, keepdims=True)          # log-sum-exp trick
        with np.errstate(invalid="ignore"):
            p = np.exp(lj - m); return p / p.sum(1, keepdims=True)
```

```
    def predict(s, X): return s.cls[s.log_joint(X).argmax(1)]

LOAN = [("High", "Salaried", "Good", 1), ("High", "Self-employed", "Good", 1), ("Medium",
"Salaried", "Good", 1),
        ("Medium", "Self-employed", "Poor", 0), ("Low", "Salaried", "Poor", 0), ("Low", "Self-
employed", "Poor", 0),
        ("Medium", "Salaried", "Poor", 1), ("Low", "Salaried", "Good", 1), ("High", "Self-employed",
"Poor", 0),
        ("Low", "Self-employed", "Good", 0)]
ENC = [{"High": 0, "Medium": 1, "Low": 2, "Very High": 3}, {"Salaried": 0, "Self-employed": 1},
{"Good": 0, "Poor": 1}]
enc = lambda rows: np.array([[ENC[j][v] for j, v in enumerate(r[:3])] for r in rows])
XL, yL = enc(LOAN), np.array([r[3] for r in LOAN]); KL = [4, 2, 2]      # 'Very High' reserved as
unseen value
nb0 = CatNB(0, KL).fit(XL, yL)
R["q3_prior"] = nb0.prior.tolist()
R["q3_cpt"] = {n: nb0.cpt[j][:, :(3 if j == 0 else KL[j])].tolist() for j, n in
enumerate(["Income(H,M,L)", "Employment(S,SE)", "Credit(G,P)"])}
qs = {"A (Low,Self-emp,Poor)": ("Low", "Self-employed", "Poor"), "B (Medium,Salaried,Good)":
("Medium", "Salaried", "Good")}
R["q3_manual"] = {}
for n, q in qs.items():
    xq = enc([q]); lj = np.exp(nb0.log_joint(xq))[0]; R["q3_manual"][n] = {"score_No": lj[0],
"score_Yes": lj[1], "post": nb0.predict_proba(xq)[0].tolist()}
xv = enc([("Very High", "Salaried", "Good")])
R["q3_mle_vh"] = np.exp(nb0.log_joint(xv))[0].tolist()
R["q3_alpha"] = {}
for a in (0.1, 1, 5):
    m = CatNB(a, KL).fit(XL, yL); R["q3_alpha"][a] = {"scores": np.exp(m.log_joint(xv))[0].tolist(),
"post": m.predict_proba(xv)[0].tolist(),
                                        "P(VH|Yes)": float(m.cpt[0][1, 3]), "P(VH|No)": float(m.cpt[0][0,
3])}
from sklearn.naive_bayes import CategoricalNB
sk = CategoricalNB(alpha=1.0, min_categories=KL).fit(XL, yL); mine1 = CatNB(1.0, KL).fit(XL, yL)
Xt = np.vstack([enc([q for q in qs.values()]), xv])
R["q3_verify"] = {"mine": mine1.predict_proba(Xt).tolist(), "sklearn": sk.predict_proba(Xt).tolist(),
```

```
            "max_abs_diff": float(np.abs(mine1.predict_proba(Xt) - sk.predict_proba(Xt)).max())}
R["q3_underflow"] = {"product": float(0.3 ** 2000), "log": 2000 * math.log(0.3)}

def gen(n, seed, flip=.05):
    """y~Bern(.5); 4 informative binary features; x5,x6 = noisy copies of x1 (redundant)."""
    r = np.random.default_rng(seed); y = r.integers(0, 2, n)
    p1 = np.array([[.25, .40, .40, .45], [.75, .60, .60, .55]])[y]
    X = (r.random((n, 4)) < p1).astype(int)
    cp = [X[:, 0] ^ (r.random(n) < flip) for _ in range(2)]
    return np.column_stack([X] + cp).astype(int), y
acc = lambda p, y: float(((p > .5).astype(int) == y).mean())
def logloss(p, y): p = np.clip(p, 1e-12, 1 - 1e-12); return float(-(y * np.log(p) + (1 - y) * np.log(1 - p)).mean())
def reliability(p, y, bins=8):
    idx = np.clip((p * bins).astype(int), 0, bins - 1); xs, ys = [], []
    for b in range(bins):
        m = idx == b
        if m.sum() >= 20: xs.append(p[m].mean()); ys.append(y[m].mean())
    return xs, ys
def split(seed, n=2000):
    X, y = gen(n, seed); k = int(.7 * n); return X[:k], y[:k], X[k:], y[k:]
Xtr, ytr, Xte, yte = split(SEED)
m4 = CatNB(1).fit(Xtr[:, :4], ytr); m6 = CatNB(1).fit(Xtr, ytr)
p4, p6 = m4.predict_proba(Xte[:, :4])[:, 1], m6.predict_proba(Xte)[:, 1]
R["q3_exp"] = {"without": {"acc": acc(p4, yte), "ll": logloss(p4, yte), "mean_conf": float(np.maximum(p4, 1 - p4).mean())},
            "with": {"acc": acc(p6, yte), "ll": logloss(p6, yte), "mean_conf": float(np.maximum(p6, 1 - p6).mean())},
            "corr_x1_x5": float(np.corrcoef(Xtr[:, 0], Xtr[:, 4])[0, 1]), "n_train": len(ytr), "n_test": len(yte)}

# ================================ Q4 ==========================================
pP, pC = .10, .05; pO = {(1, 1): .95, (1, 0): .60, (0, 1): .80, (0, 0): .02}; pA = {1: .90, 0: .05}; pS = {1: .70, 0: .01}
f = lambda pr, v: pr if v else 1 - pr
```

```
def joint(p, c, o, a, s): return f(pP, p) * f(pC, c) * f(pO[(p, c)], o) * f(pA[o], a) * f(pS[o], s)
IDX = dict(P=0, C=1, O=2, A=3, S=4)
def enum_query(q, ev):                  # inference by enumeration -> (distribution, #joint terms summed)
    res, terms = {}, 0
    for qv in (0, 1):
        tot = 0.0
        for v in itertools.product((0, 1), repeat=5):
            if v[IDX[q]] != qv or any(v[IDX[k]] != e for k, e in ev.items()): continue
            tot += joint(*v); terms += 1
        res[qv] = tot
    z = sum(res.values()); return {k: x / z for k, x in res.items()}, terms
R["q4_queries"] = {}
for name, q, ev in [("P(O=T)", "O", {}), ("P(C=T|A=T)", "C", {"A": 1}), ("P(C=T|A=T,P=T)", "C",
{"A": 1, "P": 1}),
                    ("P(C=T)", "C", {}), ("P(C=T|O=T)", "C", {"O": 1}), ("P(C=T|P=T)", "C", {"P": 1})]:
    d, t = enum_query(q, ev); R["q4_queries"][name] = {"value": d[1], "terms": t}
R["q4_manual_O_terms"] = [pP * pC * .95, pP * (1 - pC) * .60, (1 - pP) * pC * .80, (1 - pP) * (1 - pC)
* .02]
R["q4_params"] = {"bn": 1 + 1 + 4 + 2 + 2, "full": 2 ** 5 - 1}
def sample_bn(n, seed):
    r = np.random.default_rng(seed); P = r.random(n) < pP; C = r.random(n) < pC
    O = r.random(n) < np.vectorize(lambda p, c: pO[(int(p), int(c))])(P, C)
    A = r.random(n) < np.where(O, pA[1], pA[0]); S = r.random(n) < np.where(O, pS[1], pS[0])
    return np.column_stack([P, C, O, A, S]).astype(int)
data = sample_bn(1000, SEED); R["q4_est"] = {}
for n in (50, 200, 1000):
    d = data[:n]; row = {}
    for (p, c), true_p in pO.items():
        m = (d[:, 0] == p) & (d[:, 1] == c); row[f"{'TF'[1-p]}{'TF'[1-c]}"] = {"n": int(m.sum()), "est":
(float(d[m, 2].mean()) if m.sum() else None), "true": true_p}
    errs = [abs(v["est"] - v["true"]) for v in row.values() if v["est"] is not None]
    R["q4_est"][n] = {"cells": row, "mae": float(np.mean(errs)), "undefined": sum(v["est"] is None for
v in row.values()),
                      "pP": float(d[:, 0].mean()), "pC": float(d[:, 1].mean()),
                      "pA|O": [float(d[d[:, 2] == o, 3].mean()) for o in (1, 0)], "pS|O": [float(d[d[:, 2] == o,
4].mean()) for o in (1, 0)]}
```

```
fig, ax = plt.subplots(figsize=(4.6, 2.5)); ax.axis("off")
xy = {"P": (0, 1), "C": (2, 1), "O": (1, 0), "A": (0, -1), "S": (2, -1)}
lab = {"P": "P\nPower surge", "C": "C\nCoolant fail", "O": "O\nOverheat", "A": "A\nAlarm", "S":
"S\nShutdown"}
for k, (x, y) in xy.items(): ax.text(x, y, lab[k], ha="center", va="center", fontsize=7.5,
bbox=dict(boxstyle="round,pad=.4", fc="#fef3c7", ec="#b45309"))
for a, b in [("P", "O"), ("C", "O"), ("O", "A"), ("O", "S")]:
    (x1, y1), (x2, y2) = xy[a], xy[b]; ax.annotate("", (x2 * .8 + x1 * .2, y2 * .8 + y1 * .2), (x1 * .8 + x2 *
.2, y1 * .8 + y2 * .2), arrowprops=dict(arrowstyle="-|>", color="k"))
ax.set_xlim(-.6, 2.6); ax.set_ylim(-1.5, 1.5); save("q4_net")

# ================================ Q5
==========================================
npdf = lambda x, mu, var: np.exp(-(x - mu) ** 2 / (2 * var)) / np.sqrt(2 * np.pi * var)
def em_step(x, pi, mu, var):
    dens = pi * npdf(x[:, None], mu, var); ll = float(np.log(dens.sum(1)).sum())      # LL of the
incoming parameters
    r = dens / dens.sum(1, keepdims=True); Nk = r.sum(0)
    mu2 = (r * x[:, None]).sum(0) / Nk
    var2 = np.maximum((r * (x[:, None] - mu2) ** 2).sum(0) / Nk, 1e-6)
    return Nk / len(x), mu2, var2, ll, r
loglik = lambda x, pi, mu, var: float(np.log((pi * npdf(x[:, None], mu, var)).sum(1)).sum())
def em(x, pi, mu, var, tol=1e-6, maxit=1000):
    """Convergence rule: stop when |LL_t - LL_{t-1}| < tol (1e-6) or after maxit iterations."""
    lls = []
    for it in range(maxit):
        pi, mu, var, ll, r = em_step(x, pi, mu, var); lls.append(ll)
        if it > 0 and abs(lls[-1] - lls[-2]) < tol: break
    lls.append(loglik(x, pi, mu, var)); return pi, mu, var, lls, r
x7 = np.array([1.8, 2.5, 3.1, 5.0, 7.6, 8.4, 9.0])
pi0, mu0, var0 = np.array([.5, .5]), np.array([2., 8.]), np.array([1., 1.])
pi1, mu1, var1, ll0, r7 = em_step(x7, pi0, mu0, var0)
R["q5_manual"] = {"r1": r7[:, 0].tolist(), "r2": r7[:, 1].tolist(), "Nk": r7.sum(0).tolist(), "pi":
pi1.tolist(), "mu": mu1.tolist(),
                  "var": var1.tolist(), "ll_before": ll0, "ll_after": loglik(x7, pi1, mu1, var1)}
rg = np.random.default_rng(SEED); n = 500; z = (rg.random(n) < .6).astype(int)
```

```
TRUE = dict(pi=[.4, .6], mu=[2., 6.], var=[1., 2.25])
xs = np.where(z == 1, rg.normal(6, 1.5, n), rg.normal(2, 1.0, n))
runs = []
for s in range(10):
    r_ = np.random.default_rng(s); mu_i = r_.uniform(xs.min(), xs.max(), 2); v_i = np.full(2,
r_.uniform(.5, 4))
    pi_, mu_, var_, lls, _ = em(xs, np.array([.5, .5]), mu_i, v_i)
    runs.append({"seed": s, "init_mu": mu_i.tolist(), "ll": lls[-1], "pi": pi_.tolist(), "mu": mu_.tolist(),
"var": var_.tolist(), "iters": len(lls) - 1,
                 "swapped": bool(mu_[0] > mu_[1]), "monotone": bool(np.all(np.diff(lls) >= -1e-9)), "lls":
lls})
R["q5_runs"] = runs
best = max(runs, key=lambda d: d["ll"])
def fit_k(K, tries=10):
    bestk = None
    for s in range(tries):
        r_ = np.random.default_rng(100 + s)
        out = em(xs, np.full(K, 1 / K), r_.uniform(xs.min(), xs.max(), K), np.full(K, r_.uniform(.5, 4)))
        if bestk is None or out[3][-1] > bestk[3][-1]: bestk = out
    return bestk
R["q5_bic"] = {}
for K in (1, 2, 3):
    pi_, mu_, var_, lls, _ = fit_k(K); p = 3 * K - 1
    R["q5_bic"][K] = {"ll": lls[-1], "params": p, "bic": p * math.log(n) - 2 * lls[-1], "mu":
np.round(mu_, 2).tolist(), "pi": np.round(pi_, 3).tolist()}
def kmeans(x, K, r):
    mu = r.choice(x, K, replace=False)
    for _ in range(200):
        lab = np.abs(x[:, None] - mu).argmin(1); new = np.array([x[lab == k].mean() if (lab == k).any()
else mu[k] for k in range(K)])
        if np.allclose(new, mu): break
        mu = new
    lab = np.abs(x[:, None] - mu).argmin(1); return mu, lab, float(((x - mu[lab]) ** 2).sum())
km = min((kmeans(xs, 2, np.random.default_rng(s)) for s in range(10)), key=lambda t: t[2])
order = np.argsort(km[0]); mu_k = km[0][order]; lab_k = np.where(km[1] == order[0], 0, 1)
pi_k = np.bincount(lab_k) / n; var_k = np.array([xs[lab_k == k].var() for k in (0, 1)])
```

```
_, _, _, _, rb = em(xs, np.array([.5, .5]), np.array(best["init_mu"]), np.full(2, 1.))
bm = np.array(best["mu"]); lab_em = rb.argmax(1)
if bm[0] > bm[1]: lab_em = 1 - lab_em
R["q5_hard"] = {"km_mu": mu_k.tolist(), "km_pi": pi_k.tolist(), "km_var": var_k.tolist(), "km_ll":
loglik(xs, pi_k, mu_k, var_k),
            "km_acc": float((lab_k == z).mean()), "em_acc": float((lab_em == z).mean()), "em_ll":
best["ll"],
            "ambiguous": int((rb.max(1) < .9).sum())}
fig, axs = plt.subplots(1, 3, figsize=(10, 2.7))
for r_ in runs: axs[0].plot(r_["lls"], lw=.9)
axs[0].set(xlabel="iteration", ylabel="log-likelihood", title="(a) LL per EM iteration (10 inits)");
axs[0].set_ylim(best["ll"] - 400, best["ll"] + 20)
g = np.linspace(xs.min() - 1, xs.max() + 1, 400); bp, bmu, bv = np.array(best["pi"]),
np.array(best["mu"]), np.array(best["var"])
axs[1].hist(xs, 40, density=True, color="#cbd5e1"); axs[1].plot(g, (bp * npdf(g[:, None], bmu,
bv)).sum(1), "r", label="EM mixture")
axs[1].axvline(mu_k.mean(), color="g", ls="--", label="k-means boundary");
axs[1].legend(fontsize=7); axs[1].set(title="(b) Fit on 500 readings", xlabel="reading")
Ks = [1, 2, 3]; axs[2].bar(Ks, [R["q5_bic"][k]["bic"] for k in Ks], color="#6366f1");
axs[2].set(xticks=Ks, xlabel="components K", ylabel="BIC (lower = better)", title="(c) Model
selection")
axs[2].set_ylim(min(R["q5_bic"][k]["bic"] for k in Ks) - 100, max(R["q5_bic"][k]["bic"] for k in Ks)
+ 30); save("q5")

# ============================== Q6
==========================================
pac = lambda n, e, d: math.ceil((math.log(3 ** int(n) + 1) + math.log(1 / d)) / e)
R["q6_table"] = {f"eps={e},delta={d}": pac(10, e, d) for e in (.1, .05) for d in (.05, .01)}
def find_s_sim(n, m, trials, rng, tgt=(0, 1)):
    """Target: x1 AND NOT x2 (2 literals), uniform examples. Exact error = P(target) - P(h)."""
    X = rng.random((trials, m, n)) < .5; pm = X[:, :, tgt[0]] & ~X[:, :, tgt[1]]
    lp = np.all(X | ~pm[:, :, None], 1); ln = np.all(~X | ~pm[:, :, None], 1)       # literals surviving
    cnt = lp.sum(1) + ln.sum(1); dead = (lp & ln).any(1)
    return .25 - np.where(dead, 0.0, 2.0 ** -cnt)
rg = np.random.default_rng(SEED); ms = [5, 10, 20, 30, 40, 60, 80, 140, 280]; sim = {}
for m in ms:
```

```
    e = find_s_sim(10, m, 500, rg); sim[m] = {"mean_err": float(e.mean()), "frac_.1": float((e <=
.1).mean()), "frac_.05": float((e <= .05).mean())}
R["q6_sim"] = sim
vcb = lambda d, e=.1, dl=.05: math.ceil((4 * math.log2(2 / dl) + 8 * d * math.log2(13 / e)) / e)
R["q6_vc"] = {"interval": vcb(2), "halfplane": vcb(3), "rectangle": vcb(4)}
def separable(P, lab):                  # linear separability by LP feasibility
    A = [[-(2 * l - 1) * p[0], -(2 * l - 1) * p[1], -(2 * l - 1)] for p, l in zip(P, lab)]
    return linprog([0, 0, 0], A_ub=A, b_ub=[-1] * len(P), bounds=[(None, None)] * 3).status == 0
tri = [(0, 0), (1, 0), (0, 1)]; sq = [(0, 0), (1, 1), (1, 0), (0, 1)]
R["q6_shatter"] = {"halfplane_3pts": all(separable(tri, l) for l in itertools.product((0, 1), repeat=3)),
                   "halfplane_4pts_all": all(separable(sq, l) for l in itertools.product((0, 1), repeat=4))}
dia = [(0, 1), (1, 0), (0, -1), (-1, 0)]
def rect_ok(P, lab):                    # smallest axis-aligned box around positives must exclude negatives
    pos = [p for p, l in zip(P, lab) if l]
    if not pos: return True
    x0, x1, y0, y1 = min(p[0] for p in pos), max(p[0] for p in pos), min(p[1] for p in pos), max(p[1] for
p in pos)
    return not any(x0 <= p[0] <= x1 and y0 <= p[1] <= y1 for p, l in zip(P, lab) if not l)
R["q6_shatter"]["rect_4pts"] = all(rect_ok(dia, l) for l in itertools.product((0, 1), repeat=4))
R["q6_shatter"]["rect_5pts_fail"] = not all(rect_ok(dia + [(0, 0)], l) for l in itertools.product((0, 1),
repeat=5))
def interval_ok(P, lab):                # smallest interval around positives must exclude negatives
    pos = [p for p, l in zip(P, lab) if l]
    return not pos or not any(min(pos) <= p <= max(pos) for p, l in zip(P, lab) if not l)
R["q6_shatter"]["interval_2pts"] = all(interval_ok([1, 2], l) for l in itertools.product((0, 1), repeat=2))
R["q6_shatter"]["interval_3pts_fail"] = not all(interval_ok([1, 2, 3], l) for l in itertools.product((0, 1),
repeat=3))
fig, axs = plt.subplots(2, 3, figsize=(10, 5.2))
ns = np.arange(1, 101); axs[0, 0].plot(ns, [pac(n, .05, .05) for n in ns]); axs[0, 0].set(xlabel="n
(features)", ylabel="m", title="(a) m vs n  (ε=δ=0.05)")
es = np.linspace(.01, .3, 100); axs[0, 1].plot(es, [pac(10, e, .05) for e in es]); axs[0, 1].set(xlabel="ε",
ylabel="m", title="(b) m vs ε  (n=10, δ=0.05)")
ds = np.logspace(-4, -.5, 100); axs[0, 2].semilogx(ds, [pac(10, .05, d) for d in ds]); axs[0,
2].set(xlabel="δ", ylabel="m", title="(c) m vs δ  (n=10, ε=0.05)")
axs[1, 0].plot(ms, [sim[m]["mean_err"] for m in ms], "o-", label="mean true error"); axs[1,
0].axhline(.05, color="r", ls=":", label="ε=0.05")
```

```
axs[1, 0].axvline(280, color="k", ls="--", label="PAC bound m=280"); axs[1, 0].set(xlabel="m",
ylabel="error", title="(d) Find-S: error vs m"); axs[1, 0].legend(fontsize=7)
axs[1, 1].plot(ms, [sim[m]["frac_.05"] for m in ms], "s-"); axs[1, 1].axhline(.95, color="r", ls=":");
axs[1, 1].axvline(280, color="k", ls="--")
axs[1, 1].set(xlabel="m", ylabel="P(error ≤ 0.05)", title="(e) success fraction (need ≥ 0.95)")
for P, c, mk in [(dia, "tab:blue", "o")]: axs[1, 2].scatter(*zip(*P), s=60, c=c)
axs[1, 2].add_patch(plt.Rectangle((0, -1), 1, 2, fc="none", ec="r", ls="--")); axs[1, 2].text(-1.4, -1.5,
"box of {top,right,bottom} excludes left", fontsize=7)
axs[1, 2].set(title="(f) 4 points shattered by rectangles", xlim=(-1.5, 1.5), ylim=(-1.6, 1.5),
aspect="equal"); save("q6")

# ---- Part C: mistake bounds
rg = np.random.default_rng(SEED); NB = 6; Hs = np.arange(2 ** NB)              # H = monotone
conjunctions over 6 vars (64 hyps)
def halving_run(T=100):
    tgt = int(rg.integers(2 ** NB)); alive = np.ones(2 ** NB, bool); mist = 0
    for _ in range(T):
        x = int(rg.integers(2 ** NB)); preds = (x & Hs) == Hs
        vote = int(preds[alive].sum() * 2 > alive.sum()); y = int((x & tgt) == tgt)
        mist += vote != y; alive &= (preds == bool(y))
    return mist
hm = [halving_run() for _ in range(2000)]
R["q6_halving"] = {"mean": float(np.mean(hm)), "max": int(max(hm)), "bound": math.log2(64)}
def conj_learner(n, target, X):
    hyp = np.ones(2 * n, bool); mist = 0                                  # all literals: predicts negative
    for x in X:
        lit = np.concatenate([x, ~x]); pred = bool(np.all(lit[hyp]))
        y = bool(np.all(x[target])) if len(target) else True
        if pred != y: mist += 1; hyp &= lit                               # only positives cause mistakes
    return mist
n = 10; rg = np.random.default_rng(SEED); mm = []
for t in range(3000):
    tgt = np.where(rg.random(n) < rg.choice([.1, .3, .6]))[0]; pb = rg.choice([.5, .8, .95])
    mm.append(conj_learner(n, tgt, rg.random((300, n)) < pb))
adv = [np.array([j != i for j in range(n)]) for i in range(n)]
R["q6_conj"] = {"max": int(max(mm)), "mean": float(np.mean(mm)), "bound": n + 1,
```

```
            "adv": conj_learner(n, np.array([], int), [np.ones(n, bool)] + adv)}
Xd = [1, 2, 3, 4]; C = [tuple(int(x >= t) for x in Xd) for t in range(1, 6)]      # thresholds, |C| = 5
from functools import lru_cache
@lru_cache(None)
def opt(V):
   best = 0
   for i in range(len(Xd)):
      V0 = tuple(c for c in V if c[i] == 0); V1 = tuple(c for c in V if c[i] == 1)
      o = lambda S: opt(S) if S else -1e9
      val = min(max(o(V1) if pr == 1 else o(V0), 1 + (o(V0) if pr == 1 else o(V1))) for pr in (0, 1)) if
V0 and V1 else 0
      best = max(best, val)
   return best
vc = max(len(S) for r_ in range(1, 5) for S in itertools.combinations(range(4), r_) if len({tuple(c[i] for
i in S) for c in C}) == 2 ** len(S))
R["q6_opt"] = {"vc": vc, "opt": opt(tuple(C)), "log2": math.log2(len(C)), "size": len(C)}

# ================================ Q7
==========================================
class TAN:
   """Tree-augmented NB for binary features: each feature has the class + at most one feature parent
(CMI max spanning tree)."""
   def __init__(s, alpha=1.): s.a = alpha
   def fit(s, X, y):
      n, d = X.shape; a = s.a; s.d = d; s.prior = np.bincount(y, minlength=2) / n; W = np.zeros((d, d))
      for i in range(d):
         for j in range(i + 1, d):
            for c in (0, 1):
               m = y == c; cnt = np.array([[((X[m, i] == u) & (X[m, j] == v)).sum() for v in (0, 1)] for u
in (0, 1)]) + a
               P = cnt / cnt.sum(); W[i, j] += m.mean() * (P * np.log(P / (P.sum(1, keepdims=True) *
P.sum(0, keepdims=True)))).sum()
      W = W + W.T; s.par = [-1] * d; tree = {0}
      while len(tree) < d:
         i, j = max(((i, j) for i in tree for j in range(d) if j not in tree), key=lambda e: W[e]); s.par[j] = i;
tree.add(j)
```

```
        s.W = W; s.t = []
        for j in range(d):
            p = s.par[j]
            if p < 0: s.t.append(np.array([[(((X[:, j] == v) & (y == c)).sum() + a) / ((y == c).sum() + 2 * a)
for v in (0, 1)] for c in (0, 1)]))
            else: s.t.append(np.array([[[(((X[:, j] == v) & (X[:, p] == u) & (y == c)).sum() + a) / (((X[:, p]
== u) & (y == c)).sum() + 2 * a)
                                        for v in (0, 1)] for u in (0, 1)] for c in (0, 1)]))
        return s
    def predict_proba(s, X):
        lj = np.tile(np.log(s.prior), (len(X), 1))
        for j in range(s.d):
            for c in (0, 1): lj[:, c] += np.log(s.t[j][c, X[:, j]] if s.par[j] < 0 else s.t[j][c, X[:, s.par[j]], X[:, j]])
        p = np.exp(lj - lj.max(1, keepdims=True)); return p / p.sum(1, keepdims=True)
tan = TAN(1).fit(Xtr, ytr); pt = tan.predict_proba(Xte)[:, 1]
R["q7_seed42"] = {"NB4": [acc(p4, yte), logloss(p4, yte)], "NB6": [acc(p6, yte), logloss(p6, yte)],
"TAN6": [acc(pt, yte), logloss(pt, yte)]}
R["q7_parents"] = tan.par
R["q7_params"] = {"NB6": 1 + 6 * 2, "TAN6": 1 + 2 + 5 * 4}
res = {k: [] for k in ("NB4", "NB6", "TAN6")}; pool = {k: ([], []) for k in res}
for s in range(100, 120):
    a, b, c_, d_ = split(s)
    for k, p in (("NB4", CatNB(1).fit(a[:, :4], b).predict_proba(c_[:, :4])[:, 1]), ("NB6", CatNB(1).fit(a,
b).predict_proba(c_)[:, 1]),
                 ("TAN6", TAN(1).fit(a, b).predict_proba(c_)[:, 1])):
        res[k].append((acc(p, d_), logloss(p, d_))); pool[k][0].append(p); pool[k][1].append(d_)
R["q7_20seeds"] = {k: {"acc": [float(np.mean([r[0] for r in v])), float(np.std([r[0] for r in v]))],
                       "ll": [float(np.mean([r[1] for r in v])), float(np.std([r[1] for r in v]))]} for k, v in
res.items()}
import time
t0 = time.perf_counter(); [CatNB(1).fit(Xtr, ytr) for _ in range(20)]; t_nb = (time.perf_counter() - t0) /
20
t0 = time.perf_counter(); [TAN(1).fit(Xtr, ytr) for _ in range(20)]; t_tan = (time.perf_counter() - t0) /
20
R["q7_fit_time_ms"] = {"NB6": t_nb * 1e3, "TAN6": t_tan * 1e3}
fig, axs = plt.subplots(1, 3, figsize=(10, 2.9))
```

```
ax = axs[0]; ax.axis("off"); ax.set_xlim(-.5, 5.5); ax.set_ylim(-3, 1.4); ax.set_title("(a) learned TAN structure (arrows: parent→child)", fontsize=8)
ax.text(2.5, 1, "Y (class)", ha="center", bbox=dict(boxstyle="round", fc="#fee2e2", ec="r"), fontsize=8)
for j in range(6):
    ax.text(j, -1, f"x{j+1}", ha="center", bbox=dict(boxstyle="round", fc="#dbeafe", ec="b"), fontsize=8)
    ax.annotate("", (j, -.75), (2.5, .8), arrowprops=dict(arrowstyle="->", color="r", alpha=.35))
    if tan.par[j] >= 0: ax.annotate("", (j, -1.15), (tan.par[j], -1.15), arrowprops=dict(arrowstyle="->", connectionstyle="arc3,rad=.45", color="b"))
for k, c in (("NB4", "tab:green"), ("NB6", "tab:red"), ("TAN6", "tab:blue")):
    xs_, ys_ = reliability(np.concatenate(pool[k][0]), np.concatenate(pool[k][1])); axs[1].plot(xs_, ys_, "o-", c=c, label={"NB4": "NB, 4 features", "NB6": "NB, +2 redundant", "TAN6": "TAN, +2 redundant"}[k], ms=3)
axs[1].plot([0, 1], [0, 1], "k:"); axs[1].set(xlabel="predicted P(y=1)", ylabel="observed frequency", title="(b) reliability (20 seeds pooled)"); axs[1].legend(fontsize=7)
axs[2].boxplot([[r[1] for r in res[k]] for k in res], tick_labels=["NB4", "NB6", "TAN6"]); axs[2].set(ylabel="test log-loss", title="(c) log-loss over 20 seeds"); save("q7")

json.dump(R, open(os.path.join(HERE, "results.json"), "w"), indent=1, default=float)
print("done")
```